

```

#include <Keypad.h>

// ===== KEYPAD =====
const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS] = {5, 4, 3, 2};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

// ===== PIN =====
const int trigPin = 10;
const int echoPin = 11;

const int buzzer = A0;
const int mornLed = 0;
const int eveLed = 1;

const int segPins[] = {A1, A2, A3, A4, A5, 12, 13};

// ===== 7 SEG =====
byte digits[13][7] = {
  {1,1,1,1,1,1,0},{0,1,1,0,0,0,0},{1,1,0,1,1,0,1},
  {1,1,1,1,0,0,1},{0,1,1,0,0,1,1},{1,0,1,1,0,1,1},
  {1,0,1,1,1,1,1},{1,1,1,0,0,0,0},{1,1,1,1,1,1,1},
  {1,1,1,1,0,1,1},{1,1,1,0,1,1,1},{0,0,1,1,1,1,1},
  {1,0,0,1,1,1,1}
};

// ===== TIME =====
const unsigned long TIME_UNIT = 1000UL;
unsigned long intervalDay = 10 * TIME_UNIT;

```

```

unsigned long intervalNight = 14 * TIME_UNIT;

unsigned long targetTime = 0;
unsigned long prevMillis = 0;

// ===== BLINK =====
unsigned long ledPrevMillis = 0;
bool ledState = false;
const unsigned long BLINK_INTERVAL = 400;

// ===== STATE =====
String inputBuffer = "";
char currentMode = ' ';
int doseStep = 0;
bool isAlarm = false;
bool startFromEvening = false;

// ===== SETUP =====
void setup() {
    for (int i = 0; i < 7; i++) pinMode(segPins[i], OUTPUT);

    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);

    pinMode(buzzer, OUTPUT);
    digitalWrite(buzzer, HIGH);

    pinMode(mornLed, OUTPUT);
    pinMode(eveLed, OUTPUT);

    digitalWrite(mornLed, HIGH);
    digitalWrite(eveLed, HIGH);
    showDigit(8);
    delay(1000);

    digitalWrite(mornLed, LOW);
    digitalWrite(eveLed, LOW);
    showDigit(0);
}

```

```

// ===== DISPLAY =====
void showDigit(int index) {
    for (int i = 0; i < 7; i++) {
        digitalWrite(segPins[i], digits[index][i]);
    }
}

// ===== LOOP =====
void loop() {
    char key = keypad.getKey();
    if (key != NO_KEY) handleKey(key);

    if (doseStep >= 1 && !isAlarm) {
        if (millis() - prevMillis >= targetTime) {
            isAlarm = true;
        }
    }

    if (isAlarm) {
        digitalWrite(buzzer, LOW);
        showDigit(doseStep % 10);

        // ===== BLINK =====
        if (millis() - ledPrevMillis >= BLINK_INTERVAL) {
            ledPrevMillis = millis();
            ledState = !ledState;
        }

        bool isMorning;
        if (!startFromEvening)
            isMorning = (doseStep % 2 != 0);
        else
            isMorning = (doseStep % 2 == 0);

        if (isMorning) {
            digitalWrite(mornLed, ledState ? HIGH : LOW);
            digitalWrite(eveLed, LOW);
        } else {
            digitalWrite(mornLed, LOW);
            digitalWrite(eveLed, ledState ? HIGH : LOW);
        }
    }
}

```

```

    }

    long d = getDistance();

    if (d > 0 && d < 15) {
        delay(300);
        if (getDistance() < 15) {
            confirmDose();
        }
    }

} else {
    digitalWrite(buzzer, HIGH);
}
}

// ===== KEYPAD =====
void handleKey(char key) {

    if (key == '*') {
        inputBuffer = "";
        showDigit(0);
    }

    else if (key == 'A') {

        if (inputBuffer.length() > 0 && currentMode == ' ') {
            long val = inputBuffer.toInt();

            if (val > 0) {
                targetTime = val * TIME_UNIT;
                doseStep = 1;
                startFromEvening = false;
                prevMillis = millis();
                showDigit(1);
            }

            inputBuffer = "";
        }
        else {

```

```

        currentMode = 'A';
        inputBuffer = "";
        showDigit(10);
    }
}

else if (key == 'B') {

    if (inputBuffer.length() > 0 && currentMode == ' ') {
        long val = inputBuffer.toInt();

        if (val > 0) {
            targetTime = val * TIME_UNIT;
            doseStep = 1;
            startFromEvening = true;
            prevMillis = millis();
            showDigit(1);
        }

        inputBuffer = "";
    }
    else {
        currentMode = 'B';
        inputBuffer = "";
        showDigit(11);
    }
}

else if (key >= '0' && key <= '9') {
    inputBuffer += key;
    showDigit(key - '0');
}

else if (key == '#') {
    long val = inputBuffer.toInt();

    if (currentMode == 'A') {
        intervalDay = val * TIME_UNIT;
    }
    else if (currentMode == 'B') {

```

```

        intervalNight = val * TIME_UNIT;
    }

    inputBuffer = "";
    currentMode = ' ';
}

else if (key == 'D') {
    if (isAlarm) confirmDose();
}

else if (key == 'C') {
    resetSystem();
}
}

// ===== RESET =====
void resetSystem() {
    doseStep = 0;
    isAlarm = false;
    startFromEvening = false;

    digitalWrite(buzzer, HIGH);
    digitalWrite(mornLed, LOW);
    digitalWrite(eveLed, LOW);

    inputBuffer = "";
    currentMode = ' ';
    showDigit(0);
}

// ===== CONFIRM =====
void confirmDose() {
    isAlarm = false;
    digitalWrite(buzzer, HIGH);

    digitalWrite(mornLed, LOW);
    digitalWrite(eveLed, LOW);

    bool isMorning;

```

```

    if (!startFromEvening)
        isMorning = (doseStep % 2 != 0);
    else
        isMorning = (doseStep % 2 == 0);

    if (isMorning) targetTime = intervalNight;
    else targetTime = intervalDay;

    prevMillis = millis();
    doseStep++;

    if (doseStep > 60) {
        doseStep = 0;
        showDigit(12);
    } else {
        showDigit(doseStep % 10);
    }
}

// ===== SENSOR =====
long getDistance() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(echoPin, HIGH, 30000);
    if (duration == 0) return 999;

    return duration / 58;
}

```